Preshing on Programming

- Twitter
- RSS

Navigate…

- Blog
- Archives
- About
- Contact

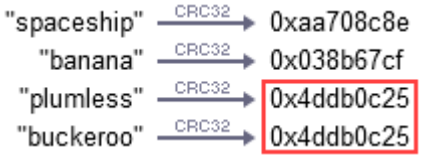
May 04, 2011

Hash Collision Probabilities

A hash function takes an item of a given type and generates an integer hash value within a given range. The input items can be anything: strings, compiled shader programs, files, even directories. The same input always generates the same hash value, and a good hash function tends to generate different hash values when given different inputs.

A hash function has no awareness of "other" items in the set of inputs. It just performs some arithmetic and/or bit-magic operations on the input item passed to it. Therefore, there's always a chance that two different inputs will generate the same hash value.

Take the well-known hash function CRC32, for example. If you feed this function the two strings "plumless" and "buckeroo", it generates the same value. This is known as a hash collision.



What is the probability of a hash collision? This question is just a general form of the birthday problem from mathematics. The answer is not always intuitive, so it's difficult to guess correctly. Let's derive the math and try to get a better feel for those probabilities.

Calculating the Probability of a Hash Collision

There are many choices of hash function, and the creation of a good hash function is still an active area of research. Some hash functions are fast; others are slow. Some distribute hash values evenly across the available range; others don't. If you're interested in the real-world performance of a few known hash functions, Charles Bloom and strchr.com offer some comparisons. For our purposes, let's assume the hash function is pretty good — it distributes hash values evenly across the available range.

In this case, generating hash values for a collection of inputs is a lot like generating a collection of random numbers. Our question, then, translates into the following:

> Given k randomly generated values, where each value is a non-negative integer less than N, what is the probability that at least two of them are equal?

It turns out it's actually a bit simpler to start with the reverse question: What is the probability that they are all unique? Whatever the answer to the reverse question, we can just subtract it from one, and we'll have the answer to our original question.

Given a space of N possible hash values, suppose you've already picked a single value. After that, there are $N-1$ remaining values (out of a possible N) that are unique from the first. Therefore, the probability of randomly generating two integers that are unique from each other is $\frac{N-1}{N}$.

After that, there are $N-2$ remaining values (out of a possible N) that are unique from the first two, which means that the probability of randomly generating three integers that are all unique is $\frac{N-1}{N} \times \frac{N-2}{N}$. (We can multiply the probabilities together because each random number generation is an independent event.)

In general, the probability of randomly generating k integers that are all unique is:

$$\frac{N-1}{N} \times \frac{N-2}{N} \times \cdots \times \frac{N-(k-2)}{N} \times \frac{N-(k-1)}{N}$$

On a computer, this can be quite slow to evaluate for large k. Luckily, the above expression is approximately equal to:

$$e^{\frac{-k(k-1)}{2N}}$$

which is a lot faster to compute. How do we know this is a good approximation? Well, it can be shown analytically, using the Taylor expansion of e^x and an epsilon-delta proof, that the approximation error tends to zero as N increases. Or, you can just compute both values and compare them. Run the following Python script with different N, and you'll get a feeling for just how accurate the approximation is:

```
import math
N = 1000000
probUnique = 1.0
for k in xrange(1, 2000):
    probUnique = probUnique * (N - (k - 1)) / N
    print k, 1 - probUnique, 1 - math.exp(-0.5 * k * (k - 1) / N)
```

Great, so this magic expression serves as our probability that all values are unique. Subtract it from one, and you have the probability of a hash collision:

$$1 - e^{\frac{-k(k-1)}{2N}}$$

Here is a graph for $N = 2^{32}$. This illustrates the probability of collision when using 32-bit hash values. It's worth noting that a 50% chance of collision occurs when the number of hashes is 77163. Also note that the graph takes the same S-curved shape for any value of N.

Simplified Approximations

It's interesting that our approximation takes the form $1 - e^{-X}$, because it just so happens that for any X that is very small, say $\frac{1}{10}$ or less:

$$1 - e^{-X} \approx X$$

In other words, the exponent makes a pretty good approximation all by itself! In fact, the smaller the X, the more accurate it gets. So for small collision probabilities, we can use the simplified expression:

$$\frac{k(k-1)}{2N}$$

This is actually a handy representation, because it avoids some numerical precision problems in the original expression. Floating point numbers are not very good at representing values extremely close to 1.

Furthermore, if you're talking about more than a handful of k, there isn't a very big difference between $k(k-1)$ and k^2. So the absolute simplest approximation is just:

$$\frac{k^2}{2N}$$

Small Collision Probabilities

In certain applications — such as when using hash values as IDs — it can be very important to avoid collisions. That's why the most interesting probabilities are the small ones.

Assuming your hash values are 32-bit, 64-bit or 160-bit, the following table contains a range of small probabilities. If you know the number of hash values, simply find the nearest matching row. To help put the numbers in perspective, I've included a few real-world probabilities scraped from the web, like the odds of winning the lottery.

Number of **32-bit** hash values	Number of **64-bit** hash values	Number of **160-bit** hash values	**Odds of a hash collision**
77163	5.06 billion	1.42×10^{24}	1 in 2
30084	1.97 billion	5.55×10^{23}	1 in 10
9292	609 million	1.71×10^{23}	1 in 100
2932	192 million	5.41×10^{22}	1 in 1000
927	60.7 million	1.71×10^{22}	1 in 10000
294	19.2 million	5.41×10^{21}	1 in 100000
93	6.07 million	1.71×10^{21}	1 in a million
30	1.92 million	5.41×10^{20}	1 in 10 million
10	607401	1.71×10^{20}	1 in 100 million
	192077	5.41×10^{19}	1 in a billion
	60740	1.71×10^{19}	1 in 10 billion
	19208	5.41×10^{18}	1 in 100 billion
	6074	1.71×10^{18}	1 in a trillion
	1921	5.41×10^{17}	1 in 10 trillion
	608	1.71×10^{17}	1 in 100 trillion
	193	5.41×10^{16}	1 in 10^{15}
	61	1.71×10^{16}	1 in 10^{16}
	20	5.41×10^{15}	1 in 10^{17}
	7	1.71×10^{15}	1 in 10^{18}

Odds of a full house in poker
1 in 693

Odds of four-of-a-kind in poker
1 in 4164

Odds of being struck by lightning
1 in 576000

Odds of winning a 6/49 lottery
1 in 13.9 million

Odds of dying in a shark attack
1 in 300 million

Odds of a meteor landing on your house
1 in 182 trillion

[Hash Table Performance Tests »]

Comments (66)

Commenting Disabled
Further commenting on this page has been disabled by the blog admin.

RCL · *502 weeks ago*

He is Charles, not Chris (Bloom) :) Thanks for great article, anyway!

Reply ▼ **1 reply** · *active 421 weeks ago*

↳ Jeff Preshing · *502 weeks ago*

Man, that's twice I did that! Good thing so many people out there know his name.

Reply

João Salada · *490 weeks ago*

Great Article! do you know any good paper that contains what you've just posted?

Reply ▼ **2 replies** · *active 421 weeks ago*

↳ Jeff Preshing · *490 weeks ago*

The math is also derived in the Wikipedia article I linked. You'll find additional references there.

Reply

↳ João Salada · *490 weeks ago*

Thank you very much sir! ;)

Reply

Mustafa · *480 weeks ago*

Hi, thanks for this lecture ,

Please can you explain what k means?

Reply ▼ **1 reply** · *active 421 weeks ago*

Jeff Preshing · *480 weeks ago*

It's the number of hashes.

Reply

Mustafa · *480 weeks ago*

thank you for reply, also:

I want know from where this equation came:

Hash collision Probability = 2^-n = 1/2^n

Please help me!

Reply ▼ **4 replies** · *active 346 weeks ago*

ToroLiu · *405 weeks ago*

2^(-n) = 2^(-1*n) = (2^(-1))^n = (1/2)^n

For example,
2^16 = 2^ (4*4) = (2^4)^4 = 16^4

Reply

Shay · *404 weeks ago*

2^(-n) = 1/(2^n), i.e., one chance in 2^n

Reply

henry · *346 weeks ago*

I am thinking the same thing. Why this article makes it so complicated? For a binary digest of size n, if your original data length is N>n bits. Then on average, 2^(N-n) sequence will produce the same digest. In this case, for each digest, 2^(N-n) sequence out of 2^N all possible sequences will cause a collision. Therefore, the collision probability will be 1/2^n.

Reply

Jeff Preshing · *346 weeks ago*

That's the probability of two hash values being equal.

This article approximates the probability of k hash values containing a match.

You're basically just looking at the special case where k = 2.

Reply

Mario · *477 weeks ago*

Great article, very well explained :)

I miss the 128bit values in the table to be complete. That'd be very useful.

Reply

Arvind Subramanian · *468 weeks ago*

Hey Jeff,

Came to your blog while reading your excellent answer to the 1 MB sorting question on stackoverflow.

I happened to also see this article of yours, and wanted to share an article I had written sometime back, which is closely related to what you have discussed here.
https://appolo85.wordpress.com/2012/07/25/some-fu...

Would love to hear your feedback!

Cheers,
Arvind

Reply

Alexander · *466 weeks ago*

Interesting generic hash collision and speed testing (Russian): http://amsoftware.narod.ru/algo2.html

Reply

Yusup · *458 weeks ago*

great stuff, keep up the good work.

Reply

krishna · *446 weeks ago*

Thank you.....
great work

Reply

sponni · *438 weeks ago*

Hi Jeff, and thank you for helping me make an informed decision about using hashes as IDs. 256-bit hashes made of 150000 input values - I think I'll be able to sleep at night :)

Although I am pretty concerned about that meteor landing on my house.

Reply

miK · *431 weeks ago*

Thanks for a nice article. However, the aspect of original message length is not covered. CRC32 collision probability for 4 byte integer vs 1.5K Ethernet packet vs 2TB drive image are the same with regard to number of hashes. But clearly, hash collision on 4 byte integer would not be a problem (ever) whereas collision on 1.5K Ethernet is minor, but no one would consider doing CRC32 on 2TB drive image for any kind of real application. Could you explain why? Thanks again.

Reply

vibneiro · *411 weeks ago*

The last table is really valuable. I've never come across one before. Thanks!

Reply

Hat Pat · *396 weeks ago*

Could you explain how you derived the Taylor Series / epsilon proof approximation from the sequence?

Reply

randy · *392 weeks ago*

Unless I'm missing something, the function 1- exp(-x) is not S-shaped. It has a slope of 1 at x=0.

Reply ▼ **1 reply** · *active 381 weeks ago*

pete.d · *381 weeks ago*

The graph is not of 1 - exp(-x). It's of 1 - exp(-x(x-1) / 2N). The square in the exponent (among other things) has a significant and obvious effect on the shape of the graph.

Reply

Roedy Green · *390 weeks ago*

I would never have guessed the math for that was so complicated. I am looking for a related formula.

Given the bit size of the hash function (or the range of the hash value), presuming it distributes perfectly randomly, feeding n items into it, what is the most likely number of collisions?

The practical question I want to solve is how big a hash function do I need to keep collisions down to a dull roar for any given application?

A related measure would be what is the average number of collisions on any given hash value -- i.e. the length of the overflow chain if you were implementing a HashMap.

Thank for all your work.

Reply ▼ **1 reply** · *active 216 weeks ago*

Steve Thomas · *216 weeks ago*

It depends what you mean by "the number of collisions". If three numbers hash to the same value, how many collisions does that count as?

However, if you ignore these cases as being negligibly infrequent, you have N elements and each of them has a probability (k-1) / N of colliding with some other element (under the approximation that all other elements have distinct values). The expectation of a binomial distribution is p*N, and the number of colliding elements is twice the number of collisions, so the expected number of collisions is:

k * (k-1) / 2N
~= k^2 / 2N

Oh, that's the same formula as before! This is because the approximation made in calculating the exponent in the real formula comes from the condition that the likelihood of having more than one collision is negligible. Under this condition, the expected number of collisions and the likelihood of a single collision are the same, since from the definition of expectation
E(x) = sum(x * p(x)) = 0 * p(0) + 1 * p(1) + ...negligible...

= p(1)

But for larger values, the discrepancy becomes clear:
If you had 150000 32bit hash values, the likelihood of at least one collision is 93% and the expected number of collisions is 2.6.

Reply

Dan Davis · *390 weeks ago*

These numbers do not seem to match up with the wikipedia article on the Birthday Problem. Who is right?

Reply ▼ **1 reply** · *active 390 weeks ago*

Jeff Preshing · *390 weeks ago*

I just checked, and I don't see any mismatch. The numbers in Wikipedia's Probability Table match the numbers in the table at the end of this post.

Can you point to a specific discrepancy?

Reply

tony m · *380 weeks ago*

Jeff,

Great article - I have rewrote your stuff in scilab - which is a matlab clone. I used the taylor expansion and I am not getting exactly the nice S shape - for large values ... and the X and Y values are a bit off...

If your familiar with Matlab or opensource Scilab - take give it a run ...

How where you able to scale your X and Y axises?

good work though...

```
-----------------------------------------------------------------------------------------
totalhashes=77163;
hashbits=32;

taylor=zeros(1,totalhashes);
N=2^hashbits;

for(k=1:totalhashes)
taylor(k) = 1 - exp(-0.5 * k * (k-1)/N) * 100;
end

k=zeros(1:totalhashes);
for(i=1:length(k))
k(i) = i;
end

scf(1)
plot(k, taylor, 'LineWidth', 1.5)
-----------------------------------------------------------------------------------------
```

Reply

tony m · *380 weeks ago*

Got it with Matlab/scilab script - I shouldn't have multiplied by 100 when doing the taylor expansion... scripts are equivalent...

Reply

Adam · *355 weeks ago*

For anyone who's a better programmer than a mathematician, here's some simplified pseudocode. Methods A through D are different ways to calculate odds, in order of decreasing complexity and accuracy.

N = 255 // Total possible hash values, like 255 for an 8-bit hash
K = 16 // Number of randomly generated integers
E = 2.71828182846 // Euler's Number constant (Exp() returns it to the Nth)

CollisionOddsA = 1 - (Exp(-K * (K - 1)) / (N * 2))) // Method A
CollisionOddsB = 1 - (E ^ (-K * (K - 1)) / (N * 2))) // Method B
CollisionOddsC = (K * (K - 1)) / (N * 2) // Method C
CollisionOddsD = (K ^ 2) / (N * 2) // Method D

Reply

Nigel · *353 weeks ago*

Can someone tell me what the chart of collisions would look like for a 48bit seed?

Reply

Min · *341 weeks ago*

Can someone tell me what the source of k means?

Reply

Stuffer · *338 weeks ago*

Here's an interactive hash collision probability calculator where you can plug in any arbitrary numbers
http://everydayinternetstuff.com/2015/04/hash-col...

Reply

Gut Check · *322 weeks ago*

This sounds a bit off, but can you explain why more complex algorithms like MD5 or even SHA-256 would have the same collision probability? It doesn't make a lot of sense. it would seem to follow that additional complexity in the algorithms themselves lead to increasing uniqueness probability, and that it's just not that reliant on the number of bits that the algorithm produces. I guess I'm questioning the middle of the S curve you have produced. I can understand the beginning and end matching up (due to limited number of total possible values), but I would think the curves might be skewed a bit (either steeper or not) based on the complexity of the computation occurring in the algorithm itself. Otherwise, why would SHA-1 "be more secure" than MD5 when they both produce 128 bits?? Ideas?

Reply ▼ **1 reply** · *active 322 weeks ago*

Jeff Preshing · *322 weeks ago*

Collision probability is related to the uniformity of the hash's distribution. Security is related to how easy it is to crack a **known** output; that is, to find some input that produces the same output. They are different things. (SHA-1 is actually 160 bits by the way.)

You can design a 160-bit hash function with a perfectly uniform distribution that is completely trivial to crack!

Reply

Ross Spencer · *301 weeks ago*

Thank you for the post and sample code. I'm trying to understand the probability of collisions better and it has helped explain things well.

The code is useful but I don't understand it completely, can you expand on why you have used (-0.5 * k ...) in the second calculation? - and if you're able, maybe expand more on the two different calculation methods used in Python? as another user above noted - for the non mathematicians!

Reply

Frank Walsh · *282 weeks ago*

How would one convert this type of calculation to something like...

Statistical Number of collisions in a dataset of X size?

Frank

Reply

René Link · *277 weeks ago*

Detailed and clear. Thanks for this great blog post. It helped me to make a decision.

Reply

Chuck Mire · *271 weeks ago*

I wrote a CRC32 program that will calculate the CRC32 value for either a file or a string. My program has a unique feature in that you can use either the normal seed value of EDB88320 Hex to compute the CRC32 value AND/OR you can use any other seed you want. IF you have a hash collision with the normal seed value, you will NOT have a hash collision on that same file or string when a different seed is used. So just keep track of more than one CRC32 value using a different seed.

My original DOS version as well as a standalone Windows executable (with source code) is available here: http://qb45.org/files.php?action=search2&keyw...

Reply ▼ **1 reply** · *active 265 weeks ago*

↳ The Mole · *265 weeks ago*

Hi Chuck,
What you have is is 2^32 possible values for the first CRC32, and then another 2^32 possibilities for the second CRC32. This gives you 2^32 * 2^32 = 2^64 possible hash values. If your file is 8 bytes long then there are also 2^64 unique bits of content which may (but probably won't) uniquely match to each of the possible hashes. If your files are longer than 8 bytes then there must be pairs of values which match to the same hash as there is no way to give them all unique values - this known as the pigeonhole principle (https://en.wikipedia.org/wiki/Pigeonhole_principle)

Reply

Bang · *266 weeks ago*

When you choose the first hash value, should there be a probability=1/N, and then when be compared with the second one, the probability that two are equal is (1/N)*((N-1)/N)?

Reply ▼ **1 reply** · *active 240 weeks ago*

↳ dingo sky · *240 weeks ago*

No. The OP starts with picking the 2nd value ("suppose you've already picked a single value"). Had the OP started with the first pick, the statement would read something like "there are N values and the probability of generating (picking) a unique integer is N/N = 1. So you could put N/N at the beginning of the sequence, but it is obviously not necessary.

Note the first pick probability is N/N, not 1/N because you are picking any value, not a specific, predetermined value (in which case the probability would be 1/N).

Reply

Hopeful · *236 weeks ago*

can you tell how the approximation is done to that "e" formula you mentioned? Please help me.

Reply

Andrew · *236 weeks ago*

Odds of dying in a shark attack are about 1 in 30.
Odd of dying from a shark attack in a single year are about 1 in 300,000,000

Reply

Elliott Balsley · *230 weeks ago*

Great article. One thing I don't understand is how to find k. Does it depend on the number of files, or only the total size of all files being transferred? For example if I am transferring 100MB of files, I want to know the probability of a hash collision using various algorithms.

Reply **6 replies** · *active 230 weeks ago*

Jeff Preshing · *230 weeks ago*

If you are hashing each file, k would be the number of files. The size of the files is irrelevant.

Reply

Elliott Balsley · *230 weeks ago*

So you're saying that the probability of a collision is the same whether you have ten 100MB files or ten 1KB files? That doesn't seem right. Surely the probability goes up as the data size increases.

Reply

Jeff Preshing · *230 weeks ago*

Yes, I'm saying that the probability of a collision is the same whether you have ten 100MB files or ten 1KB files.

The size of a hash value (say, 32 bits) is the same for a 100MB file as it is for a 1KB file. And you are testing for collisions between hash values, not files.

Reply

Elliott Balsley · *230 weeks ago*

Okay thanks. So let me see if I have this straight. Let's say I have 8 million files on 2 disks and I want to make sure they match, so I compare MD5 hashes for each file. The probability of at least one of these files being corrupt but still having a correct hash is like this: k = 8 million; N = 2^129.

1 - [e^(8000000^2*(-1)/2^129)] = 1E-25.

But MD4 is also 128-bit. So would it have the same probability?

Reply

Jeff Preshing · *230 weeks ago*

I think you're looking at a different problem. You're asking for the probability of at least one file being corrupt, but still having the same hash. That's going to take a different formula. In particular, it depends on the probability of having a corrupt file at all (which, intuitively, probably does depend on the size of the file).

Reply

Elliott Balsley · *230 weeks ago*

Ah yes I should have phrased that better. Suppose I know that one byte in one of these 8 million files is wrong. I'm trying to figure the probability of the checksum not catching it.
Then ultimately, I would multiply that by the probability of having a corrupt file in the first place, but that piece is harder to calculate, because it depends on many variables like disk type, connection interface, filesystem, etc.

Reply

Ross · *213 weeks ago*

I can find at least 3 different occurances in the last 20 years of a meteor landing on a house, so i think your odds on that are incorrect

Reply **1 reply** · *active 213 weeks ago*

Jeff Preshing · *213 weeks ago*

I found that statistic online somewhere, so who knows? But in any case, you can't really compare the odds for a specific house with the odds for all the houses in the world. That's like comparing the odds for a single lottery ticket with the odds for millions of lottery tickets.

Reply

dingosky · *212 weeks ago*

Thanks, Jeff, this is an excellent article.

As the title suggests, the focus is on understanding the risk of collision in the generation of some number of hashes using an existing hashing algorithm. The core concepts of this analysis can be extended to another related programming need, namely, the generation of random strings. Unfortunately the common solution to this problem is to specify the creation of strings of a given length, which is both underspecified and misguided. Expanding on the ideas of this article, I created EntropyString (https://github.com/EntropyString) which creates cryptographically strong random strings by specifying the number of strings to be generated and an associated risk of a collision. The analysis (contained in the README of any of the four implemented languages) basically takes up where this article ends and explains why you should care about the entropy represented by random strings, since entropy is directly related to string collision, rather than string length.

Reply

Jordy · *208 weeks ago*

I tried to calculate the chances that k=3 random hashes are all unique for a checksum with 1 bit, i.e. N=2 possible hash values.

With the long formula I get a chance of x = ((2-(3-2))/2) * ((2-(3-1))/2) = 0
I would have expected x > 0
Is my expectation wrong?

Reply **2 replies** · *active 208 weeks ago*

Jeff Preshing · *208 weeks ago*

The probability is zero because there is no way to have three unique 1-bit values.

Reply

Jordy · *208 weeks ago*

Of course your are right. It seems I thought about the expectation "what are the chances to get three values of same kind" instead of "three values of different kind", which is not applicable here. Sorry. Note to myself: Never program at midnight after a long day of mind numbing work.

Reply

Tom · *197 weeks ago*

What kind of poker are you getting the odds from, 5 card stud, draw or texas holdem. I wonder if I will ever get a full house and have a meteor hit my house at the same time.

Reply

Naveen · *187 weeks ago*

Sorry,I would like to find out why the total number of hashes (output) is 2^32 for a 32 bit hash. My understanding is that a 32 bit hash has like 32 character. Assuming that they are alphanumeric each character has 36 different possibilities. Therefore should it not be 36^32.

Reply **4 replies** · *active 187 weeks ago*

Jeff Preshing · *187 weeks ago*

A bit is not a character. A bit has only 2 possible values.

Reply

Naveen · *187 weeks ago*

So what are these possible numbers. it is not like a binary system where there are only 1s and 0s. Because the example above on buckeroo does not give me the impression of this. like if it's aplhanumeric then why would there only be 2 possible values for each character.

so sorry. I got very confused with this.

Reply

Jeff Preshing · *187 weeks ago*

You're confusing the input and the output. CRC32 is a function that takes a string as input, and returns a 32-bit integer as output. The length of the input string doesn't matter. "buckeroo" has 8 characters, but even if we passed a 5000-character string to CRC32, it will still return a single 32-bit integer. 2^32 is the number of different possible return values from CRC32.

Reply

Naveen · *187 weeks ago*

I forgot that the output for a 32 bit string would only have 8 characters. Thanks so much for the clarifications and the prompt replies. Really appreciate it. Ur article was very interesting and was what got me interested in hashes. Great job.

Reply

aclassifier · *182 weeks ago*

I have a problem described at [1].

But I don't understand the figure at "Small Collision Probabilities". What is meant by "number of hash values" that with _less_ values gives a lesser and lesser probability of a hash collison?

I have read some of the comments and though about it for a day without any success.

If I have a CRC made from some data set and my data set would only ever make 100 different CRC values, and I have another data set that would make 1000 CRC values possible, then, by the reasoning I am able to do I would either think that the latter was the better or that they would be equal. Not that the first were best, so this can't be what's meant by "umber of hash values"? I have a big problem here!

[1] https://stackoverflow.com/questions/49885413/what...

Reply ▼ **1 reply** · *active 181 weeks ago*

↳ Jeff Preshing · *181 weeks ago*

Let's rephrase hash collision probabilities like this: "Given k randomly generated values, each selected (independently) from a set of N possible values, what is the probability that at least two of them are equal?" In this post, the set of possible values is the set of *all* integers from 0 to N - 1..... but the answer is the same for any set of size N.

In your case, if there are exactly 1000 possible values that can be chosen from, that mean N is 1000... not k. So that might be tripping you up -- a confusion between N and k. Also, make sure that scenario described here -- choosing k independent values, then checking if any of them are the same -- actually applies to your problem (I'm not sure if it does).

Reply

aclassifier · *181 weeks ago*

Thanks a lot!

So if I have a data set of 20 bytes (five 32-bit words) and in addition a CRC32 (N=2exp32) and it turns out that I would, from the possible data set, have k=77163 different hash values (CRCs) then there would be a 50% chance that one CRC32 would yield OK when the data has a fault? I get (77163 * 77162) / (2*4294967296) = 0,69 not 0,5? And this is about one per 2exp(32/2) = 2exp16=65K (found that formula somewhere).

Or is the reasoning that the 20 bytes (160 bits) could be seen as having 2exp160 combinations and even then I would expect 50% probability of a collision after 77163 messages? If so, is this the other side of saying that N doesn't matter? (I almost didn't dare to write that paragraph!)

Another thing. If one CRC8 wraps the above message again, will that count as a single 32+8=40 bit CRC, so the 50% is at about 2exp20?

Reply

Check out **Plywood**, a cross-platform, open source C++ framework:

Plywood

Recent Posts

- How C++ Resolves a Function Call
- Flap Hero Code Review
- A Small Open Source Game In C++
- Automatically Detecting Text Encodings in C++
- I/O in Plywood
- A New Cross-Platform Open Source C++ Framework
- A Flexible Reflection System in C++: Part 2
- A Flexible Reflection System in C++: Part 1
- How to Write Your Own C++ Game Engine

- Can Reordering of Release/Acquire Operations Introduce Deadlock?
- Here's a Standalone Cairo DLL for Windows
- Learn CMake's Scripting Language in 15 Minutes
- How to Build a CMake-Based Project
- Using Quiescent States to Reclaim Memory
- Leapfrog Probing
- A Resizable Concurrent Map
- New Concurrent Hash Maps for C++
- You Can Do Any Kind of Atomic Read-Modify-Write Operation
- Safe Bitfields in C++
- Semaphores are Surprisingly Versatile

Copyright © 2021 Jeff Preshing - Powered by Octopress